ACME-ADCS: Implementation Guide

Step-by-step setup of an ACME (RFC 8555) server backed by Active Directory Certificate Services, hosted on IIS, with a non-domain test client. Based on glatzert/ACME-Server-ADCS.

Overview

The ACME server is the only machine that talks to the CA. The client never authenticates to AD, so domain join isn't needed.

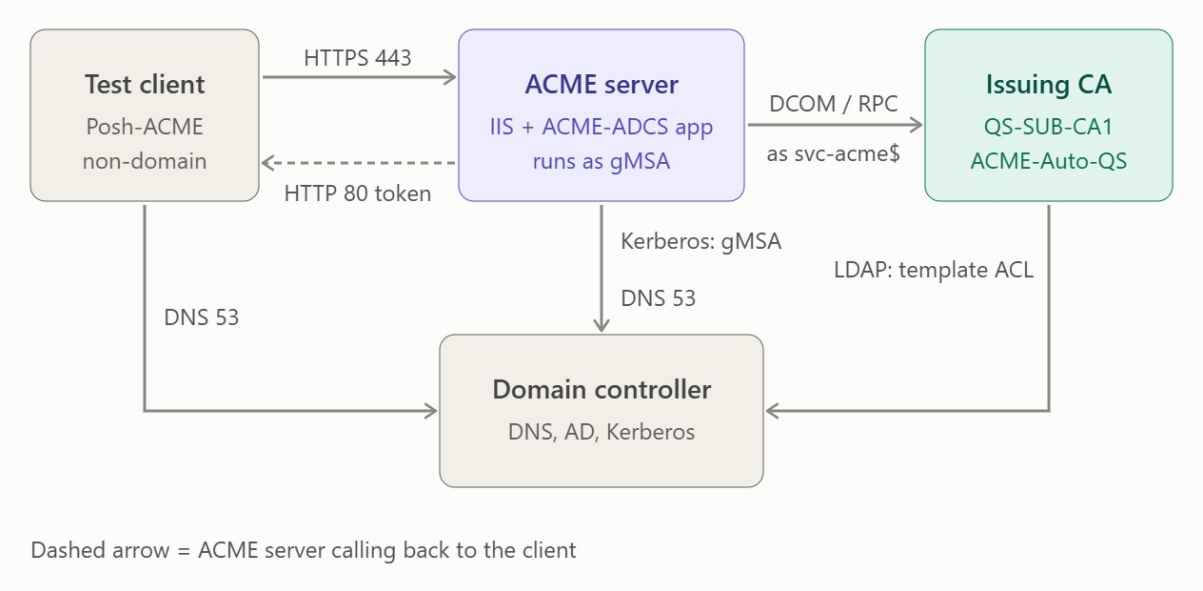


Figure 1. *Components and the protocol on each link*

The second diagram follows one request in order, from the non-domain test client through to the issued certificate.

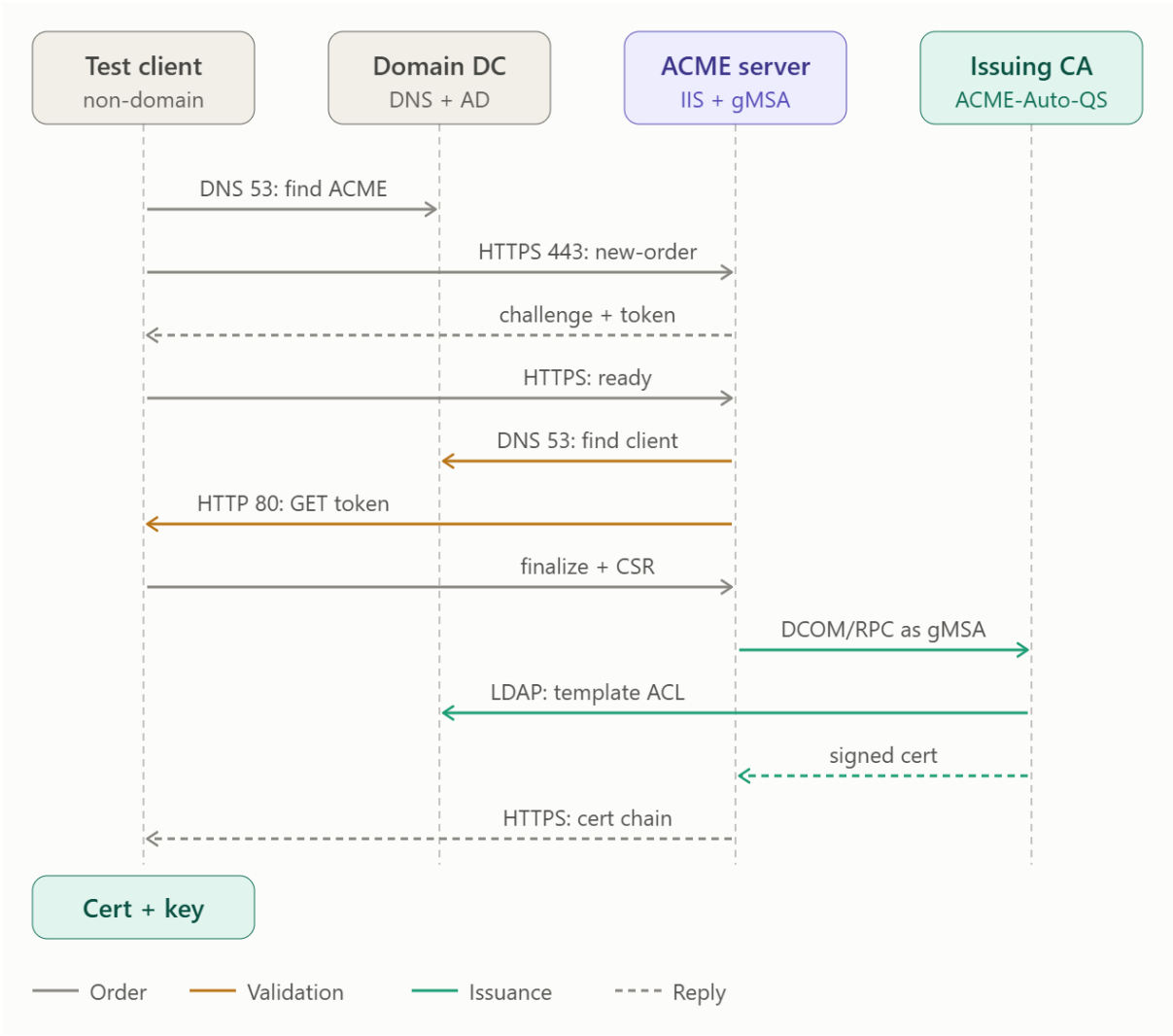


Figure 2. *Request flow from the non-domain test client to the issued certificate*

Each phase and the process responsible:

Phase	What happens	Protocol and process
Order	The client finds the ACME server by name, trusts its HTTPS certificate (root CA imported), registers an account and creates an order.	DNS 53, HTTPS 443; Posh-ACME signs each request (JWS)
Validation	The ACME server proves the client controls the name. It resolves the client's name, then fetches the token file from the client.	DNS 53, HTTP 80; Posh-ACME WebSelfHost serves the token
Issuance	The client sends a CSR. The ACME server submits it to the CA as `svc-acme$`. The CA checks the template's permissions in AD, signs the request and returns the certificate.	DCOM/RPC (ICertRequest2), Kerberos for the gMSA, LDAP for the template ACL
Delivery	The ACME server hands the signed certificate chain back to the client.	HTTPS 443

Phase 1: Identity (AD)

1. Create the gMSA

Run on a domain controller or admin workstation.

```
New-ADServiceAccount -Name svc-acme -DNSHostName qs-ra-acme.quantumspace.com
`
    -PrincipalsAllowedToRetrieveManagedPassword 'QS-RA-ACME$'
```

2. Install the gMSA on the ACME server

```
Install-WindowsFeature RSAT-AD-PowerShell
Install-ADServiceAccount svc-acme
Test-ADServiceAccount svc-acme   # must return True
```

Reboot the ACME server afterwards. If the test returned False, re-run it after the reboot.

Phase 2: CA side (certificate template)

3. Create the template

On the CA, open `certtmpl.msc`, duplicate **Web Server**, then set:

- **Compatibility:** Windows Server 2016 or later (unlocks ECDSA/CNG).
- **Cryptography:** Key Storage Provider; allow RSA and ECDSA.
- **Subject Name:** *Supply in the request*.
- **Issuance Requirements:** no manager approval.
- **Security:** `svc-acme$` gets **Read, Enroll, Autoenroll**.

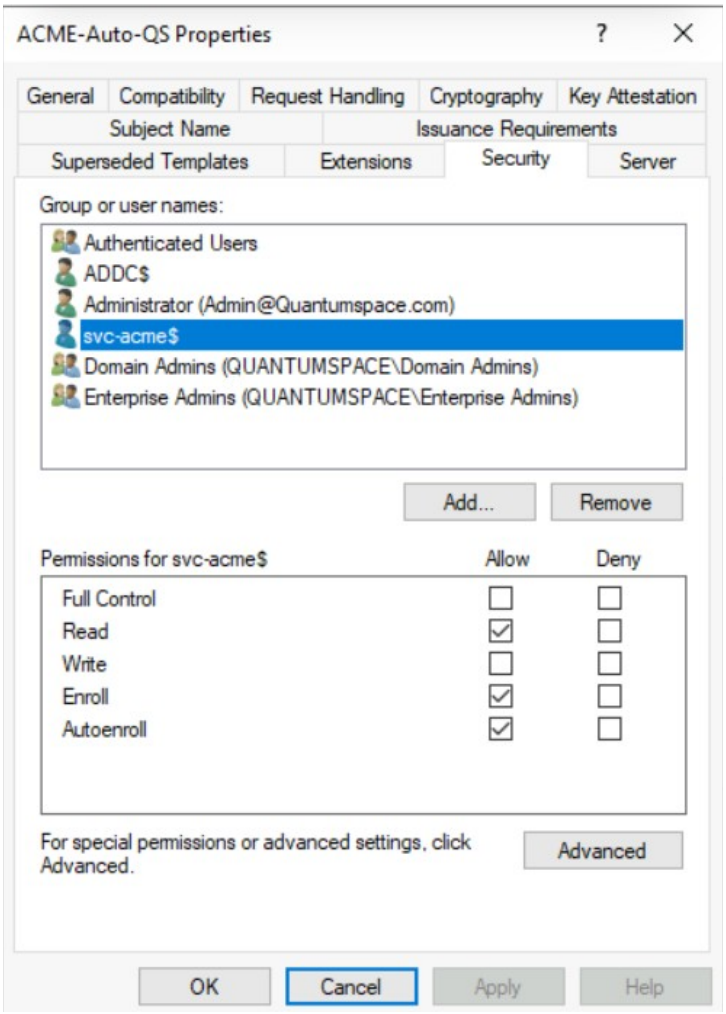


Figure 3. *Certificate template properties*

> **Note:** The Autoenroll permission on `svc-acme$` is required by the project for the process identity. It is not Group Policy autoenrollment, and clients never contact the CA.

4. Publish the template

CA console → Certificate Templates → New → Certificate Template to Issue. An unpublished template can't be requested, even with correct permissions.

5. Check the CA is reachable from the ACME server

```
certutil -config "QS-SUB-CA1.Quantumspace.com\Quantumspace Issuing CA1" -
ping
```

This is the exact CA name used in the app configuration (step 12). If the ping fails, the config will fail too: check for a name typo, firewall (RPC/DCOM) or CA permissions.

Phase 3: ACME server (IIS + .NET)

6. Add the gMSA to IIS_IUSRS

```
Add-LocalGroupMember -Group IIS_IUSRS -Member 'QUANTUMSPACE\svc-acme$'
```

This gives the app pool identity the standard IIS permissions.

7. Install IIS, then the .NET Hosting Bundle (in that order)

```
Install-WindowsFeature Web-Server,Web-Http-Logging,Web-Request-Monitor,Web-
Http-Tracing,Web-Filtering,Web-IP-Security,Web-Mgmt-Console
```

Then install the **.NET 8 (or 10) LTS Hosting Bundle** and run `iisreset`.

8. Get an HTTPS certificate for the ACME server

In `certlm.msc`, Request New Certificate, with CN/DNS = `qs-ra-acme.quantumspace.com`. ACME clients only talk HTTPS, and a certificate from your own CA is trusted by every client that trusts your root.

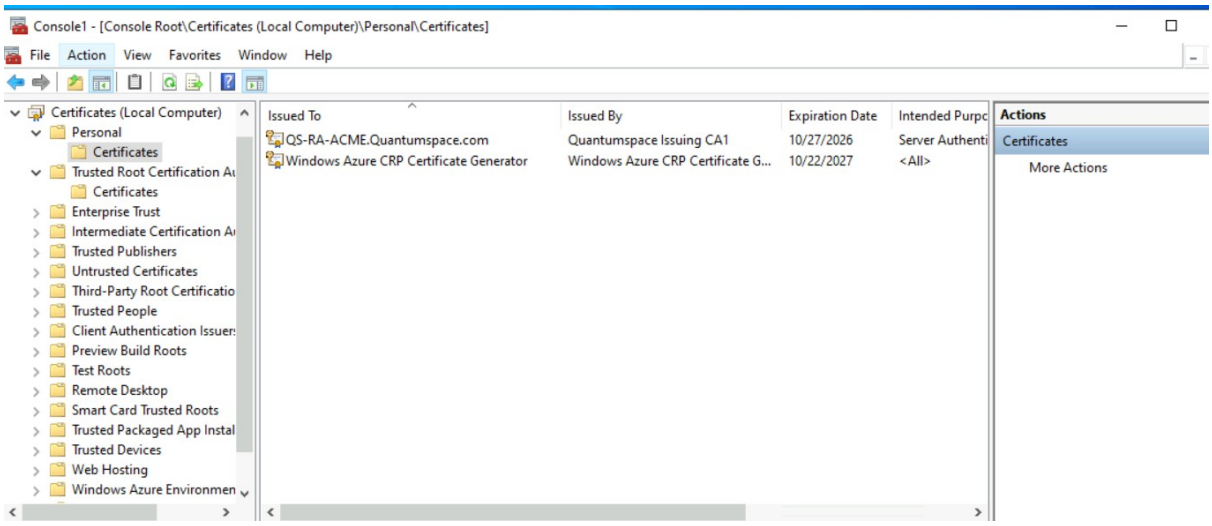


Figure 4. Server certificate in the local computer store

9. Deploy the app files

- Download the **release zip** (not "Source code") from the project's GitHub Releases: glatzert/ACME-Server-ADCS.
- Extract it directly into `C:\inetpub\acme` so `ACMEServer.ADCS.dll` and `web.config` sit at the top level.

```
Get-ChildItem C:\inetpub\acme -Recurse | Unblock-File
```

A downloaded zip carries a block flag, so unblock the extracted files. If the files end up in a sub-folder, `web.config` won't find them.

10. Create the data folder and set permissions

```
New-Item -ItemType Directory C:\ACME-ADCS -Force
```

```
icacls C:\ACME-ADCS /grant 'QUANTUMSPACE\svc-acme$:(OI)(CI)M'
icacls C:\inetpub\acme /grant 'QUANTUMSPACE\svc-acme$:(OI)(CI)RX'
```

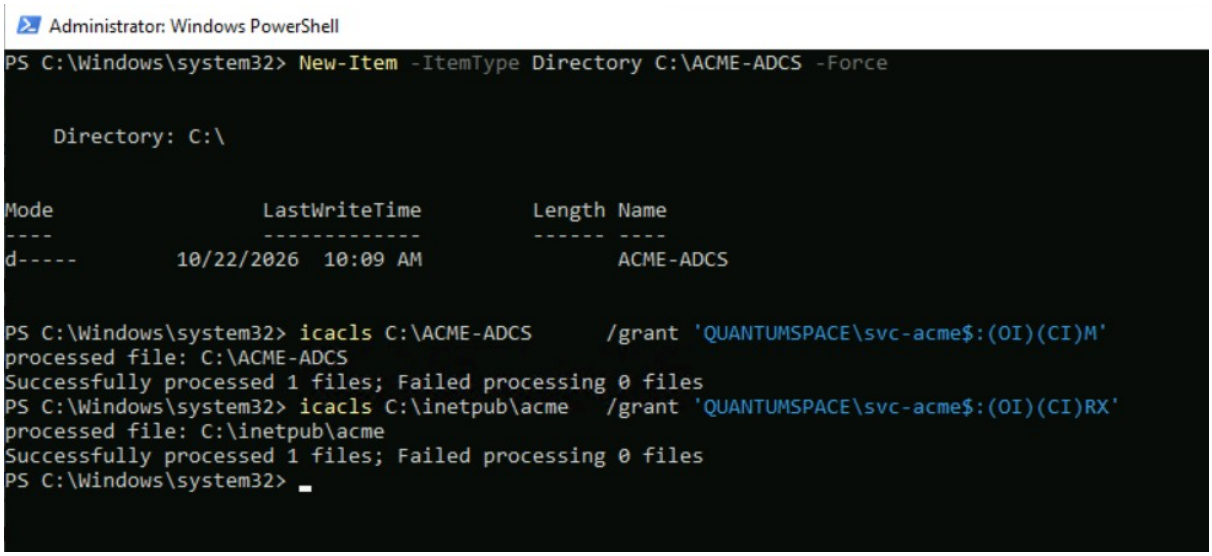


Figure 5. *Folder permissions for the gMSA*

The app writes accounts, orders and challenges to `C:\ACME-ADCS` (Modify) but only reads its own program folder (Read and execute). Keeping data outside the web folder is safer.

11. Create the app pool and site

- **App pool** `ACME-ADCS`: No Managed Code; identity = custom account `QUANTUMSPACE\svc-acme$` (leave the password blank).
- **Site** `ACME-ADCS`: path `C:\inetpub\acme`, that pool, HTTPS binding on 443 with host `qs-ra-acme.quantumspace.com` and the certificate from step 8.
- Stop the Default Web Site if it holds port 443.

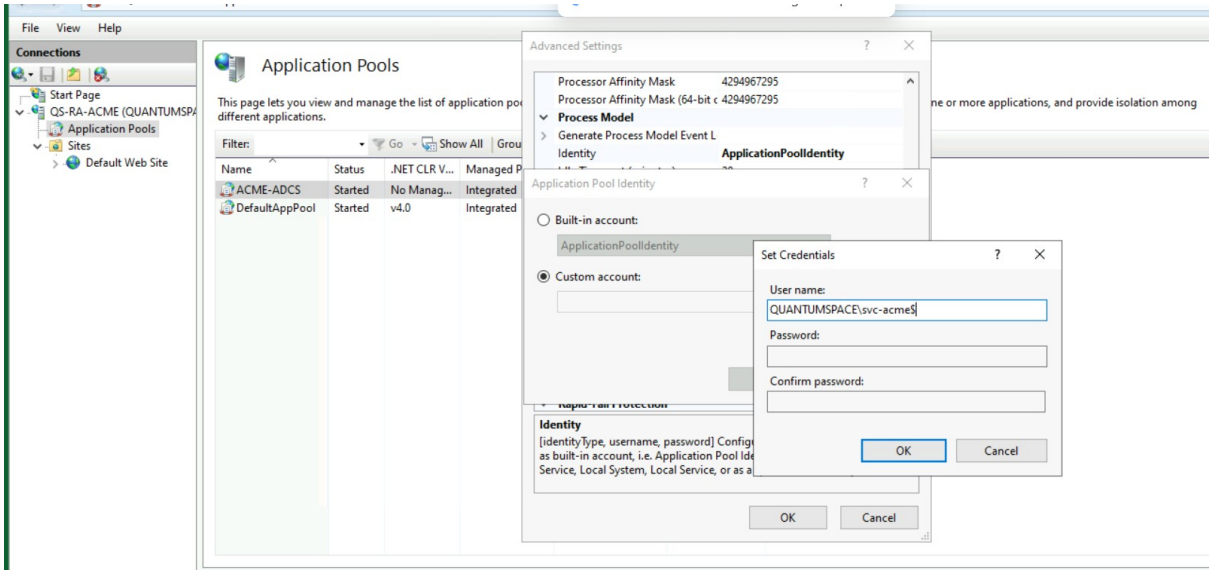


Figure 6. *IIS application pool settings*

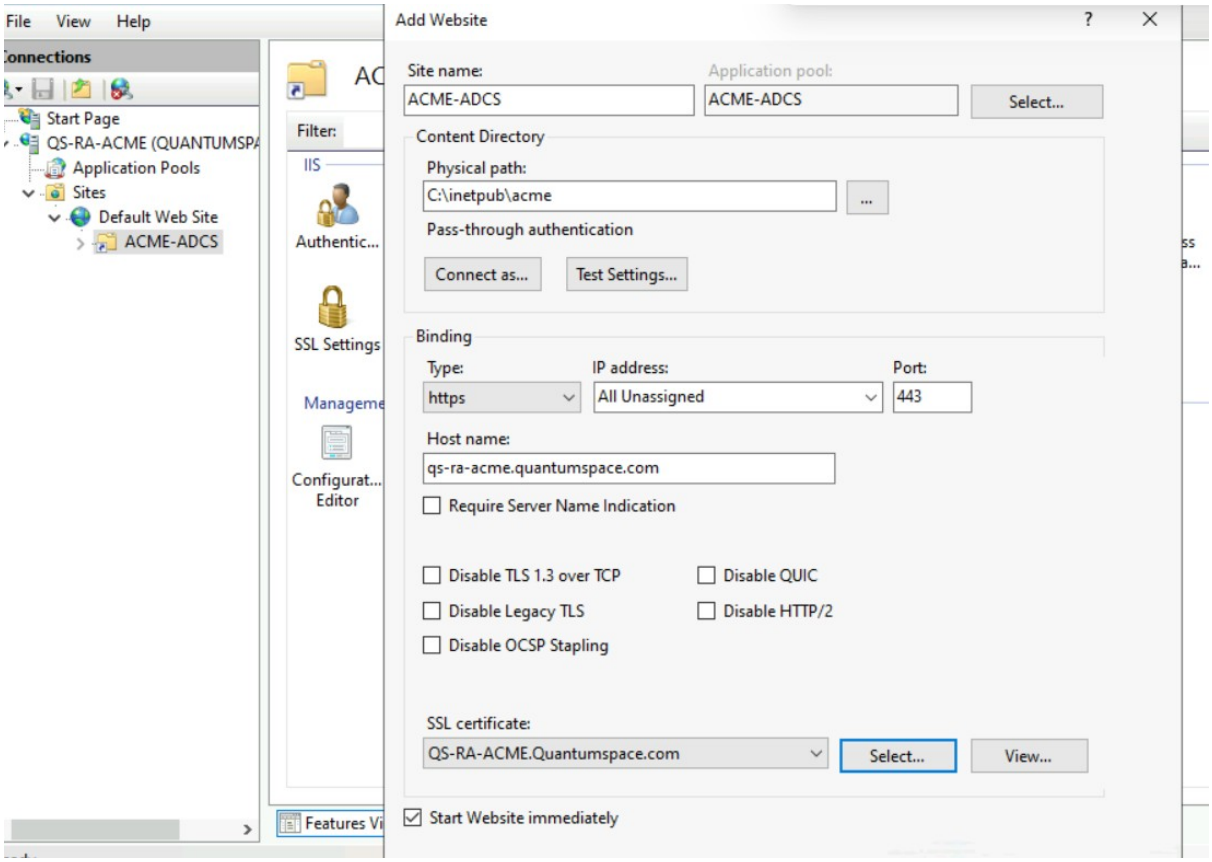


Figure 7. IIS site bindings

Why these settings:

- No Managed Code: ASP.NET Core brings its own runtime.
- The pool identity decides *who* talks to the CA, so it must be the gMSA.
- A host-name binding makes IIS answer for the name clients use.

Verify:

```
Get-IISAppPool ACME-ADCS | Select Name, State
Get-Website ACME-ADCS | Select Name, State, PhysicalPath
(Get-ItemProperty IIS:\AppPools\ACME-ADCS).processModel | Select
identityType, userName
Get-WebBinding -Name ACME-ADCS
```

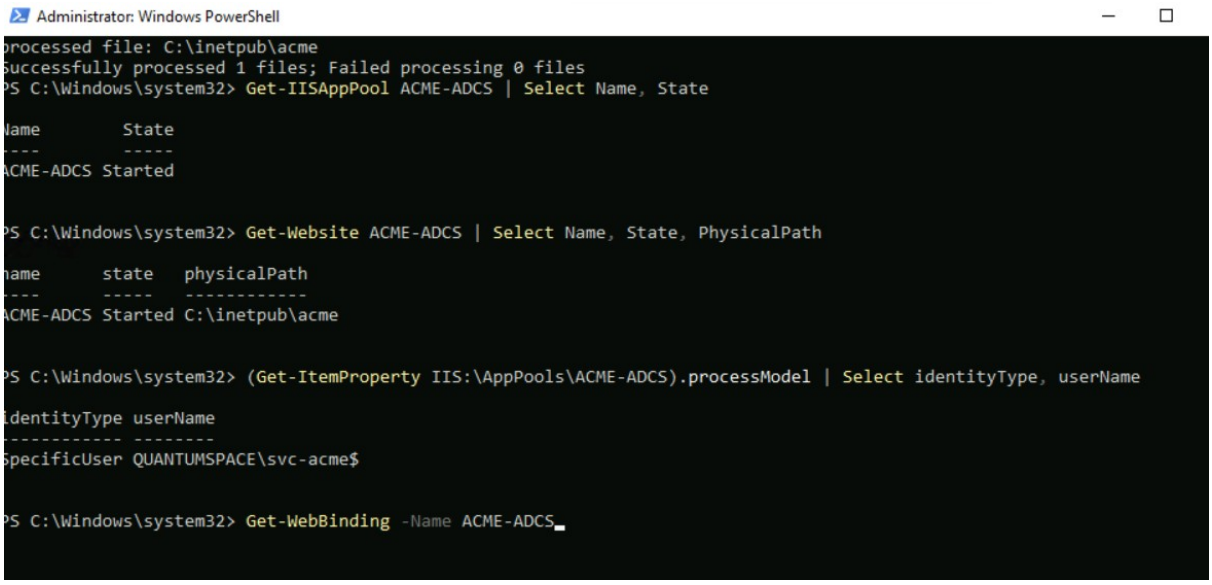


Figure 8. *Verification output for pool, site and binding*

Phase 4: Configure ACME-ADCS

12. Create appsettings.Production.json

v3.0 reads this file, not `appsettings-custom.json`.

```
@'
{
  "AcmeFileStore": { "BasePath": "C:\\ACME-ADCS\\" },
  "AcmeServer":    { "SupportsRevocation": false },
  "Profiles": {
    "Default-Profile": {
      "SupportedIdentifiers": [ "dns" ],
      "ADCSOptions": {
        "CAServer": "QS-SUB-CA1.Quantumspace.com\\Quantumspace Issuing CA1",
        "TemplateName": "ACME-Auto-QS"
      }
    }
  }
}
'@ | Set-Content C:\inetpub\acme\appsettings.Production.json -Encoding UTF8
```

13. Test the app outside IIS first

```
cd C:\inetpub\acme
dotnet .\ACMEServer.ADCS.dll
```

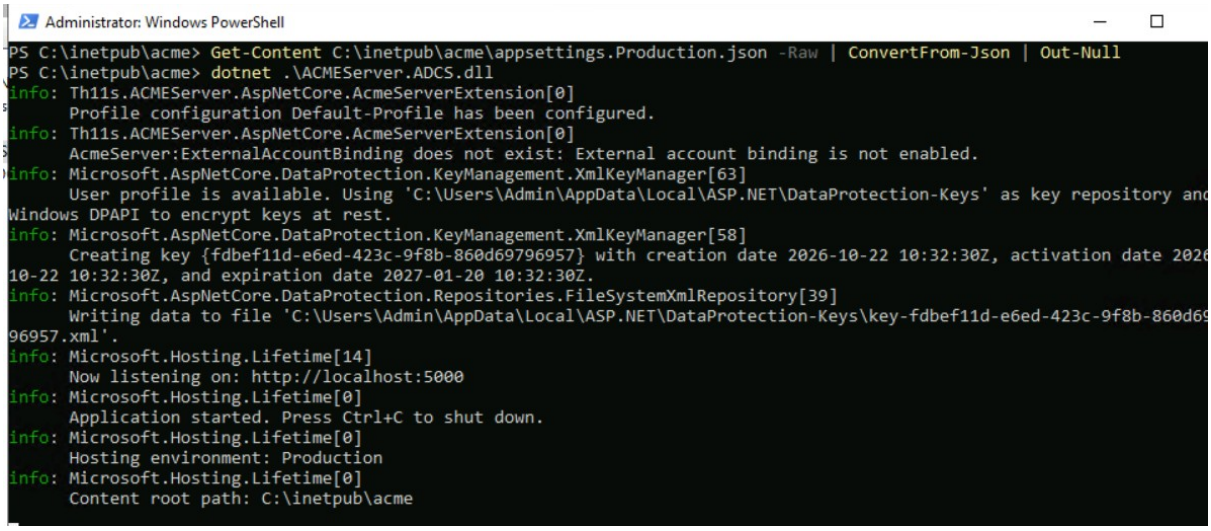


Figure 9. ACME app started from the console

14. Start IIS and check the directory

```
Restart-WebAppPool ACME-ADCS
```

Browse to `https://qs-ra-acme.quantumspace.com/`. You should see JSON containing `newNonce`, `newAccount`, `newOrder` and your profile.

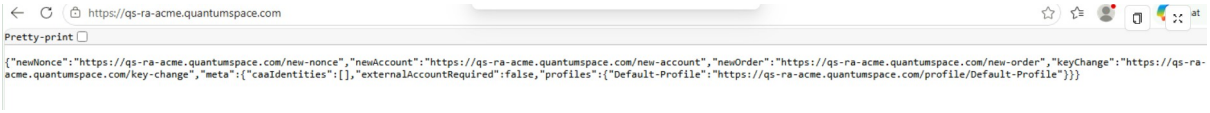


Figure 10. ACME directory JSON in the browser

> **Checkpoint:** this single page proves that IIS, the Hosting Bundle, the gMSA pool, the config and TLS all work.

Phase 5: Test client and issuance

> **Note:** The client does not need to be domain-joined. It only talks ACME over HTTPS and answers a challenge over HTTP; it never authenticates to AD or the CA. Only the ACME server needs a domain identity (the gMSA), because it performs the CA enrollment. The template is set to *Supply in the request*, so no AD information is taken from the client either.

What the client does need

1. **Trust your root CA.** Import the root certificate into the client's trusted root store, otherwise it rejects the ACME server's HTTPS certificate. (On Linux, point certbot at the PEM with `REQUESTS_CA_BUNDLE`.)
2. **Resolve the ACME server's name.** Use your internal DNS server, or add a hosts entry for `qs-ra-acme.quantumspace.com`.
3. **Be resolvable from the ACME server.** For http-01, the ACME server connects to the name in the certificate request, so `QS-TEST-SRV.quantumspace.com` (or whatever name you request) must

resolve to the client's IP from the ACME server. A non-domain machine won't register itself in DNS, so create the A record manually.

4. **Be reachable on port 80 from the ACME server**, with a web server listening there and any firewall or NSG rules in place.

15. Prepare the client's web server for validation

```
Install-WindowsFeature Web-Server
New-NetFirewallRule -DisplayName "HTTP 80 inbound (ACME)" -Direction Inbound
-Protocol TCP -LocalPort 80 -Action Allow
```

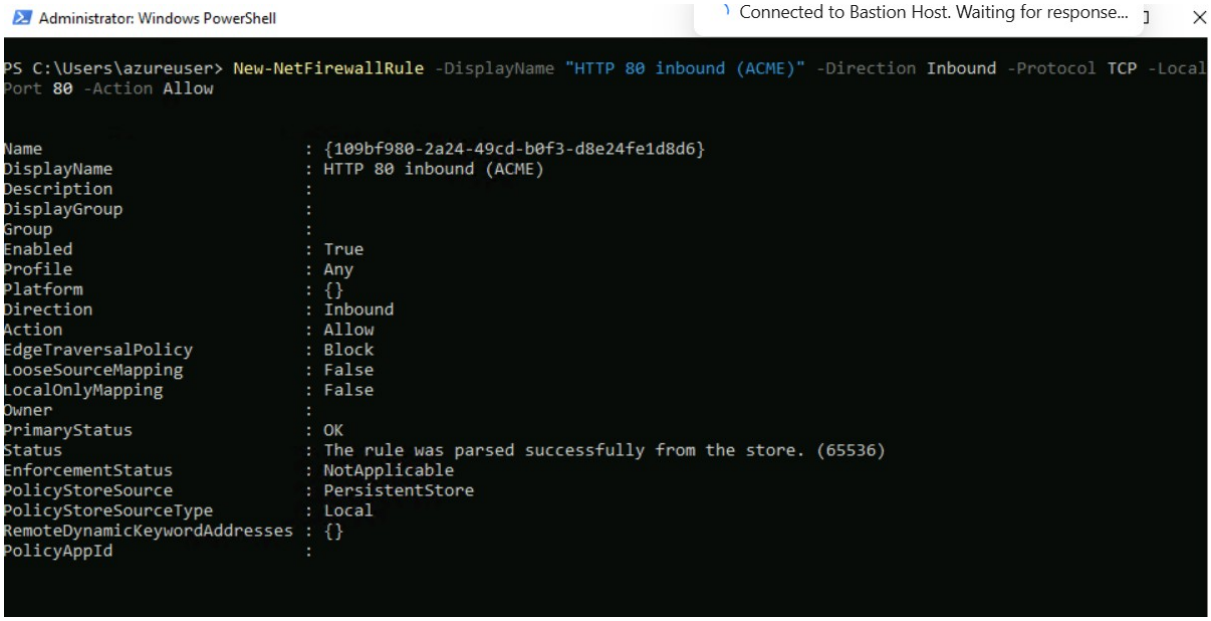


Figure 11. *Client web server and firewall rule*

Also allow inbound TCP 80 in any Azure NSG from the ACME server's IP, and add an extensionless-file mapping:

```
$dir='C:\inetpub\wwwroot\.well-known\acme-challenge'
New-Item -ItemType Directory $dir -Force
@'
<?xml version="1.0" encoding="UTF-8"?>
<configuration><system.webServer><staticContent>
  <mimeMap fileExtension="." mimeType="text/plain" />
</staticContent></system.webServer></configuration>
'@ | Set-Content "$dir\web.config" -Encoding UTF8
```

With http-01, the ACME server connects back to the client on port 80 and reads a token file. The client therefore needs a web server on port 80, firewall and NSG rules that allow it, and IIS configured to serve a file with no extension (otherwise it returns 404).

16. Request a certificate from a non-domain machine

Using Posh-ACME on the client:

```
Install-Module Posh-ACME -Scope CurrentUser
```

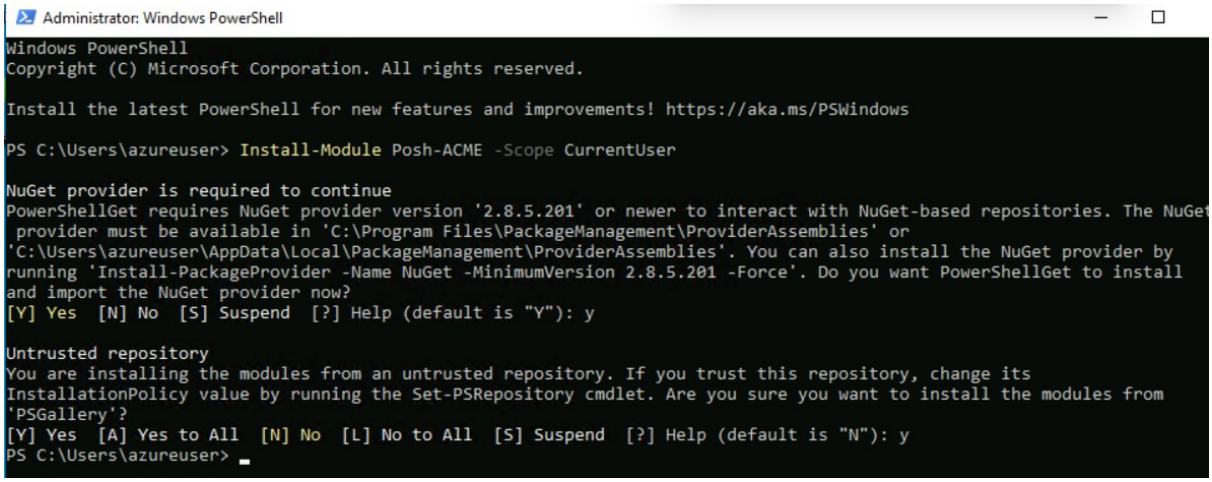


Figure 12. Posh-ACME installation

Confirm the client can resolve the ACME server:

```
Resolve-DnsName qs-ra-acme.quantumspace.com
Get-DnsClientServerAddress -InterfaceAlias Ethernet* -AddressFamily IPv4
```

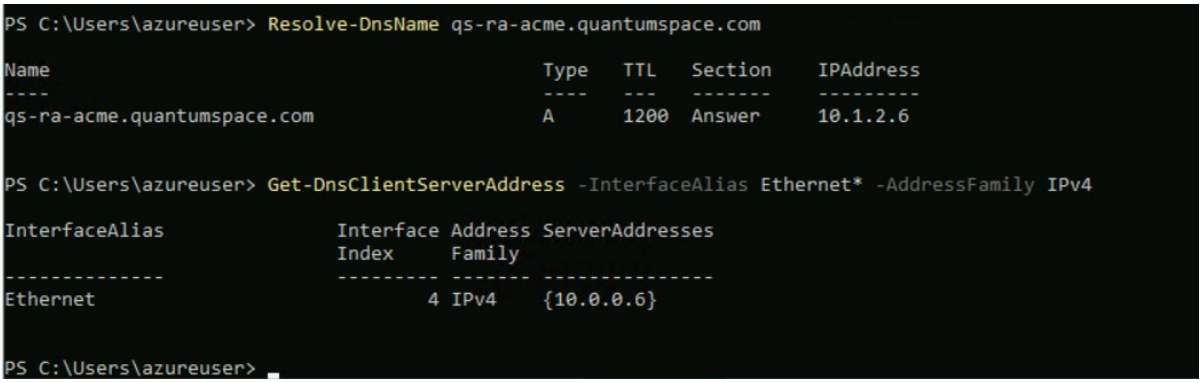


Figure 13. DNS resolution from the client

Point Posh-ACME at the directory. If this works, the directory loads and you are ready to request the certificate. A certificate or TLS error means the root CA is not trusted (requirement 1).

```
Set-PAServer -DirectoryUrl https://qs-ra-acme.quantumspace.com/
```

Because the client is not domain-joined, create its DNS A record on the DC and check that the ACME server can resolve it:

```
Add-DnsServerResourceRecordA -ZoneName quantumspace.com -Name QS-TEST-SRV -
IPv4Address <client-ip>
```

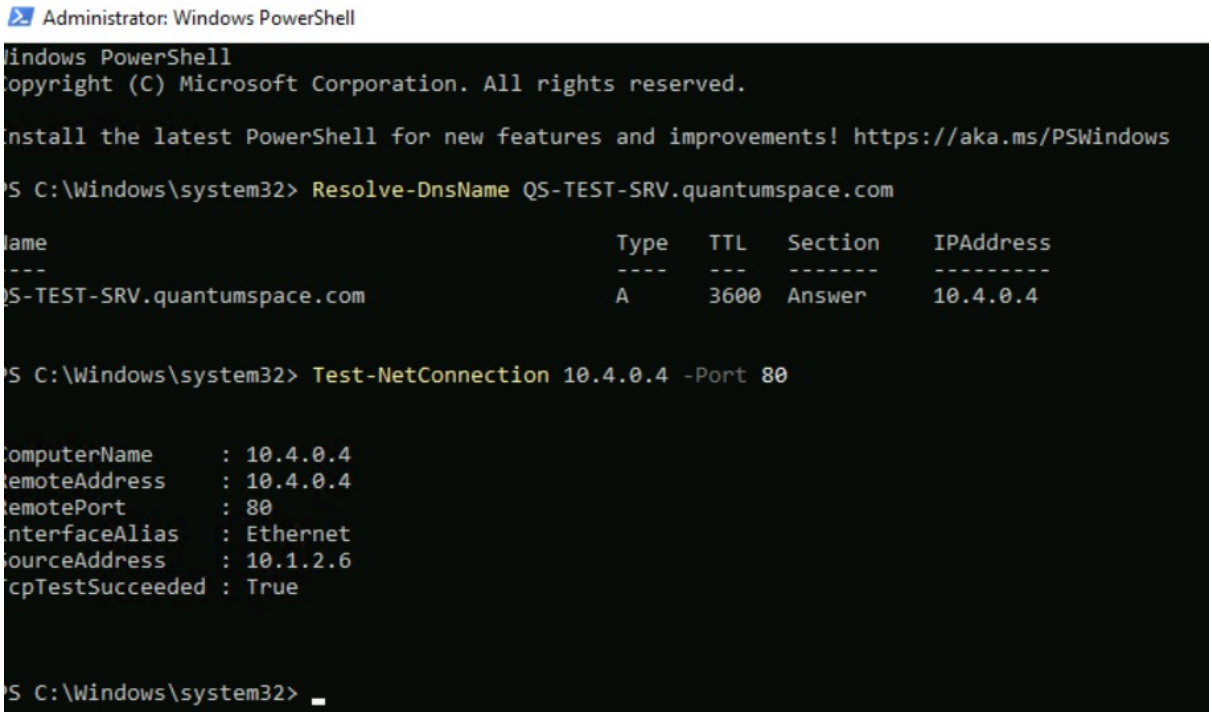


Figure 14. DNS A record for the test machine

Request the certificate:

```
New-PACertificate QS-TEST-SRV.quantumspace.com -AcceptTOS -Plugin
WebSelfHost -CertKeyLength 3072 -Force
```

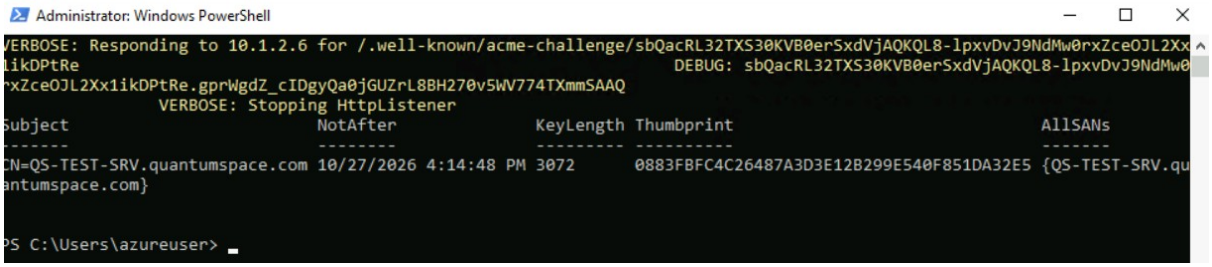


Figure 15. Certificate request output

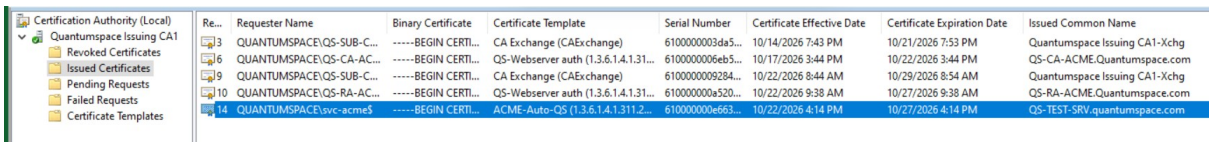


Figure 16. Certificate request result

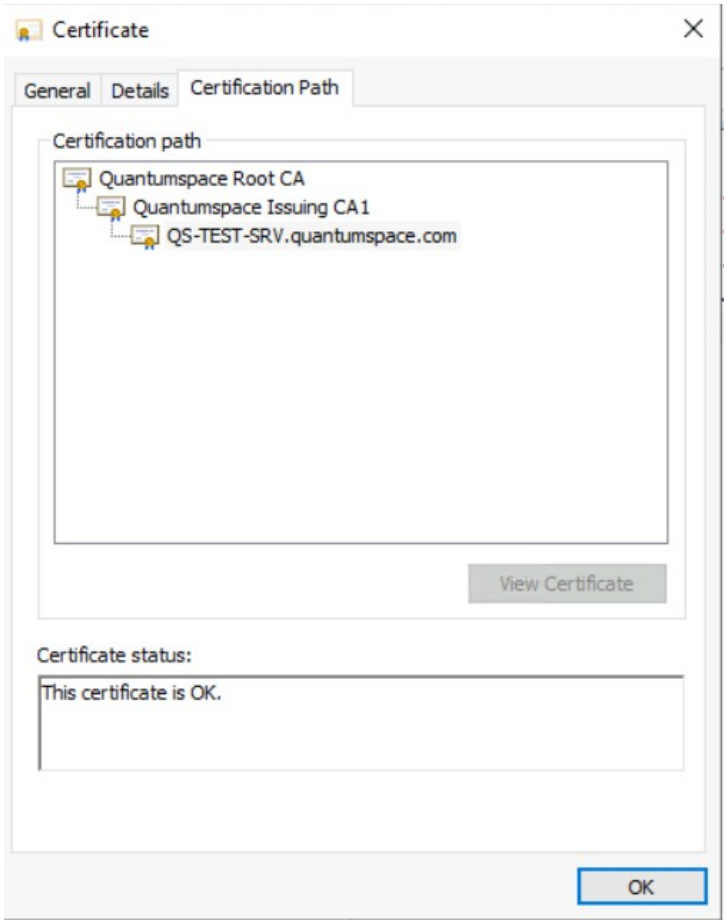


Figure 17. *Issued certificate*

Troubleshooting

Error 1: ERR_CERT_AUTHORITY_INVALID in the browser

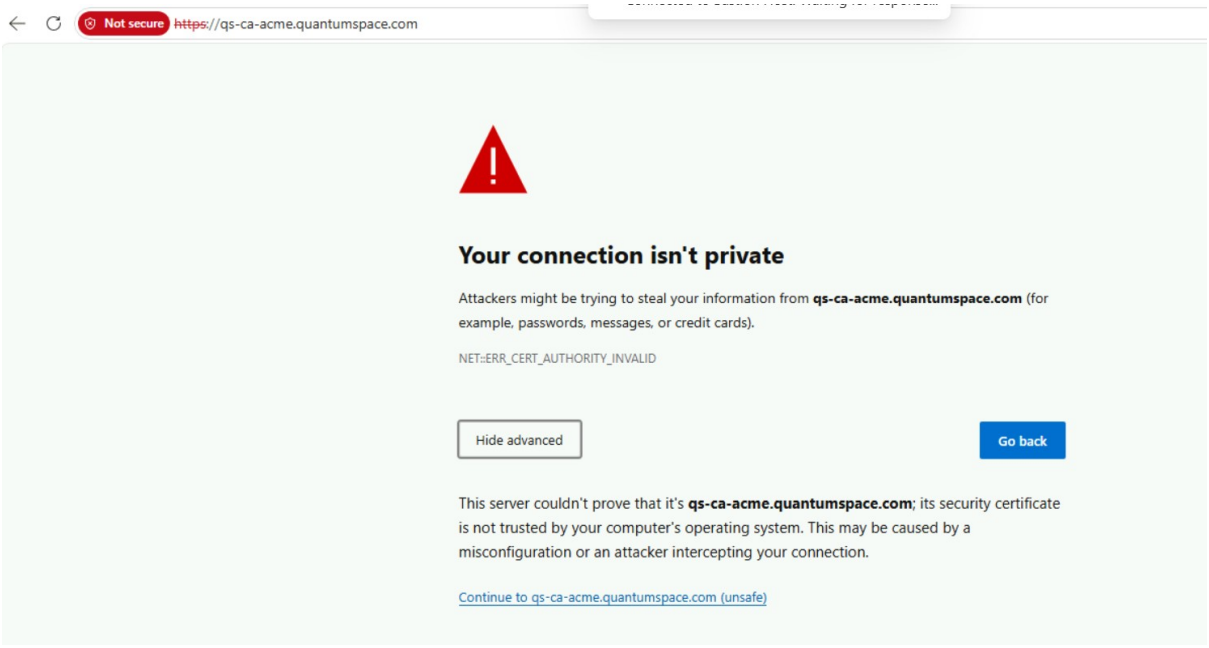


Figure 18. Browser certificate error

The browser reached the ACME server and completed the TLS handshake. This error means the client doesn't trust the CA that issued the server's HTTPS certificate. A non-domain machine doesn't receive the root CA from Group Policy, so install it manually.

Error 2: Challenge validation returns 404 (IIS on the test machine)

```
Submit-ChallengeValidation : Authorization invalid for QS-TEST-SRV.quantumspace.com: Got non 200 status code: NotFound
At C:\Users\azureuser\Documents\WindowsPowerShell\Modules\Posh-ACME\4.35.0\Public\New-PACertificate.ps1:284 char:9
+         Submit-ChallengeValidation
+         ~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : OperationStopped: (Authorization i... code: NotFound:String) [Submit-ChallengeValidation
   ], RuntimeException
    + FullyQualifiedErrorId : Authorization invalid for QS-TEST-SRV.quantumspace.com: Got non 200 status code: NotFoun
   d,Submit-ChallengeValidation

PS C:\Users\azureuser> Get-NetTCPConnection -LocalPort 80 -State Listen | Select LocalAddress, OwningProcess

LocalAddress OwningProcess
------------ -------------
::                       4


PS C:\Users\azureuser> Get-Service W3SVC, WAS -ErrorAction SilentlyContinue

Status   Name               DisplayName
------   ----               -----------
Running  W3SVC              World Wide Web Publishing Service
Running  WAS                Windows Process Activation Service


PS C:\Users\azureuser> _
```

Figure 19. IIS service state on the test machine

IIS is running on the test machine (W3SVC and WAS are Running, and PID 4 is HTTP.sys, which IIS uses). When the ACME server requests the challenge file, IIS answers with its own 404, because Posh-ACME's temporary listener can't take port 80 while IIS holds it.

Fix: stop IIS temporarily, request the certificate, then start it again.

```
iisreset /stop
# run New-PACertificate (step 16)
iisreset /start
```

Review: build order and checkpoints

Use this as a final checklist. Each step has a checkpoint to verify before moving to the next one.



Figure 20. *Build order with a verification checkpoint for each step*